

Création et utilisation de Custom Steps

Yuri López de Meneses
9 janvier 2025 v.2.0

1 Introduction

Matrox Design Assistant permet de créer des nouveaux steps, appelés Custom Steps, pour ajouter des fonctionnalités qui n'existent pas dans DA, telles que des algorithmes spécifiques d'analyse d'image, l'intégration d'autres caméras ou capteurs 3D, des algorithmes d'analyse de données, l'intégration de hardware de communication ou des actuators, etc.

Ce document explique comment développer ces custom steps pour la version plus récente (DA9) et fournit des informations additionnelles à la documentation Matrox que nous avons collecté par nos expériences passées.

Table des matières

1	Introduction	1
2	Installation de logiciels	2
3	Démarrage	2
3.1	Création d'un nouveau projet VS	2
3.2	Configuration du fichier *.das	3
3.3	Considérations de design du custom step	4
4	Programmation du corps custom step	4
4.1	Gestion d'images	4
4.1.1	Remplissage d'images	5
4.1.2	Lecture d'images	5
4.1.3	Calibration	5
4.2	Gestion d'erreurs	6
5	Programmation de l'interface graphique du custom step	6
5.1	Inputs à liste déroulante	7
5.2	Inputs désactivables ([AvailabilityConstraint])	7
5.3	Listes dynamiques	8
5.4	Inputs optionnels (None)	8
6	Documentation QuickAccess	8
7	Utilisation d'un custom step dans DA	9
7.1	Design time vs Runtime	10
7.2	Débogage d'un custom step	10
8	Actualisation d'un custom step de version précédente	10
	Références bibliographiques	11

A Troubleshooting	12
A.1 Error : Dispose was not called on a <i>StepX</i> instance	12
A.1.1 Solution	12
A.2 Error : Error loading assembly	12
A.2.1 Solution	12

2 Installation de logiciels

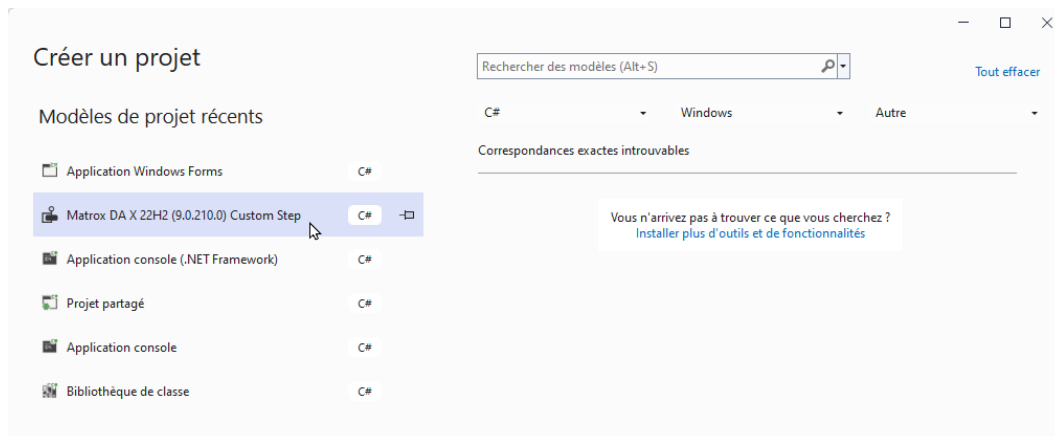
- Installer Microsoft Visual Studio 2022, en version Community (<https://visualstudio.microsoft.com/downloads/>) ou Professional (<https://azureforeducation.microsoft.com/devtools>). Il suffira d'installer les options pour C# et plateforme Windows.
- Installer le Custom Step Wizard, qui permettra de créer des projets Visual Studio avec la structure nécessaire pour des Custom Steps. Le wizard pour DA 9.0 et VS2022 est disponible dans C:/Program Files/Matrox Imaging/DA 9.0/CustomSteps/v2022/CustomStepWizard.vsix et dans un dossier analogue pour DA10.

3 Démarrage

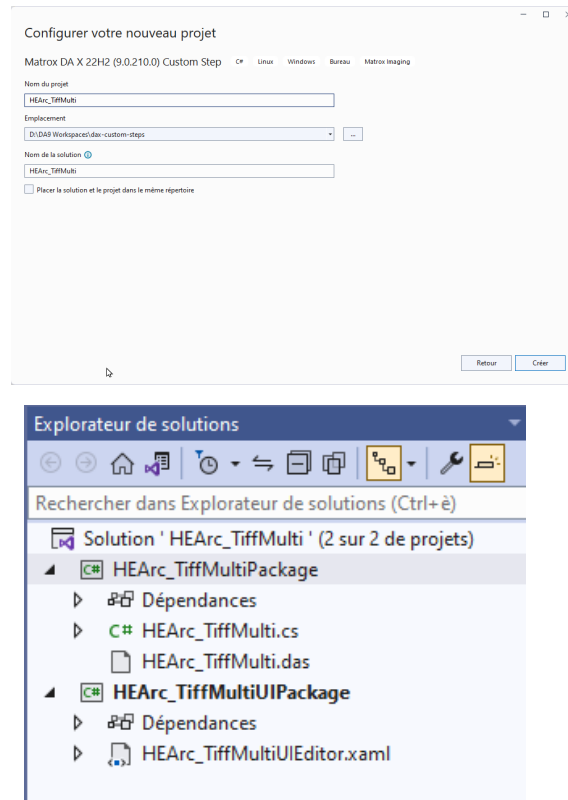
Si vous avez accès au gitlab [igib/shared/dax-custom-steps](https://gitlab.com/igib/shared/dax-custom-steps), vous pouvez créer une copie du dossier **HEArc_Example** et le renommer selon les instructions de son `readme.md` pour avoir un modèle de départ. Si non, vous devrez suivre les instructions de la section qui suit.

3.1 Création d'un nouveau projet VS

1. Lancer VS2022, cliquer sur "Créer un projet" et chercher tout en bas de la liste "Custom Step Wizard for Matrox".



2. Entrer les données du projet VS :
 - Nom de projet : `<Company>_<Nom>` où Nom est un nom bref et explicatif du step. Le nom de la société est typiquement HEArc, mais il pourrait être un nom de projet ou du client.
Exemple : `HEArc_TiffMulti`
 - Emplacement : Votre répertoire développement. On vous encourage à utiliser un gestionnaire de versions comme [Git](https://git-scm.com/) pour ce répertoire.
 - Nom de la solution : En principe le même nom que le projet. C'est le nom de la solution qui sera utilisé comme type de step dans DA, et la solution peut contenir plusieurs projets (voir ci-dessous.)



3. A ce stade vous trouverez une solution Visual Studio, avec le nom `<Company>_<Nom>` que vous avez choisi, et qui contiendra 2 projets :
 - Un projet `<Company>_<Nom>Package` qui contient la partie qui s'exécute en mode runtime.
 - Un projet `<Company>_<Nom>UIPackage` qui contient la partie d'interface utilisateur nécessaire en mode développement.
4. Pour vérifier que tout est en ordre, compilez la solution en cliquant sur **Menu→Générer→Générer la solution** (Compile en anglais)

```
Sortie
Afficher la sortie à partir de : Build
L'opération de génération a démarré...
1)----- Début de la génération : Projet : HEArc_TiffMultiPackage, Configuration : Debug Any CPU -----
1>HEArc_TiffMultiPackage -> D:\DA9 Workspaces\dax-custom-steps\HEArc_TiffMulti\HEArc_TiffMulti\assets\HEArc_TiffMultiPackage.dll
2)----- Début de la génération : Projet : HEArc_TiffMultiUIPackage, Configuration : Debug Any CPU -----
2>HEArc_TiffMultiUIPackage -> D:\DA9 Workspaces\dax-custom-steps\HEArc_TiffMulti\HEArc_TiffMulti\assets\HEArc_TiffMultiUIPackage.dll
***** Build : 2 réussite(s), 0 échec(s), 0 à jour, 0 ignorée(s) *****
```

Si ce n'est pas le cas, vérifiez les étapes précédentes ou les actualisations indiquées dans la section 8.

3.2 Configuration du fichier *.das

La prochaine étape est la configuration du fichier *.das, que vous pouvez éditer dans la solution Visual Studio. Le document [1], qui fait partie de votre installation DA, donne plus de détails.

1. Vous pouvez modifier la description du step dans la balse `<description>`.
2. Dans la balise `<step-package>` vous pouvez ajouter un bloc comme ci-dessous, pour définir le nom abrégé du step (celui indiquant le type de step et qui apparait dans le flowchart), la couleur du texte et sa couleur d'arrière plan

```
<abbreviation>
<text>TIF</text>
<background>red</background>
```

```
<textcolor>Green</textcolor>
</abbreviation>
```

3. Si votre custom step fait appel à des ressources externes, comme des icônes, fichiers de données, des DLLs, etc. elles doivent être placées dans le sous-dossier **assets**, indiquée dans la balise

```
<assetsRelativeFolder path="assets"/>
```

pour que ces ressources soient copiées lorsque le projet sera déployé.

3.3 Considérations de design du custom step

Le projet runtime (celui avec le nom du step) contiendra le coeur du step, avec ses inputs, outputs et variables. Le projet *UIPackage contient le layout (en XAML) de l'interface graphique pour configurer le step (dans la fenêtre en bas à gauche de DA.)

Le step est instancié lorsqu'il est ajouté au projet DA. C'est la seule fois que le constructeur sera appelé et il doit contenir les initialisations nécessaires. Ensuite la méthode `Run()` sera appelée à chaque fois qu'on passe par le step. La méthode `Reset()` est appelée lors qu'on presse l'icône reset dans DA. Le destructeur est appelé seulement quand on ferme le projet dans DA.

- Si l'on souhaite un step qui peut opérer en différents modes (p.ex. Read vs Write) et qu'on préfère avoir une seule classe (type de step), la méthode `Run()` devra utiliser un `switch{}` pour gérer les modes.
- Si le step doit accéder à des ressources ouvertes dans une autre instance (p.ex. envoyer un message par un canal ouvert dans un autre step du même type), il faut utiliser des membres statiques pour accéder aux mêmes ressources entre instances différentes.

4 Programmation du corps custom step

Pour des instructions plus détaillées, voir le document [1]. Pour les différentes classes et fonctions disponibles, consulter le document [2].

4.1 Gestion d'images

Si le step crée des images il faudra l'indiquer avec l'attribut `[ResponsibleOfMILAllocations(false)]` et faire l'override des méthodes `MILAllocate()` et `MILFree()`.

```
protected override void MILAllocate()
{
    _image = new Image(); //We next allocate an Image buffer from MIL.
    _image.Alloc2d(Context.MILApplication.SystemHost.Id, numcolumns, numlines,
        Image.ImageTypeConstants.Unsigned8Bit, MIL.M_IMAGE + MIL.M_DISP + MIL.M_PROC);
}
```

Lorsqu'on alloue ou crée une image, il faut le faire avec les attributs `MIL.M_IMAGE + MIL.M_DISP + MIL.M_PROC` pour indiquer qu'il s'agit d'une image, pour display et pour processing.

```
protected override void MILFree()
{
    if (_image != null)
    {
        _image.Free(); // Free MIL buffer
        _image.Dispose();
        _image = null;
    }
}
```

```
}  
if (cal != MIL.M_NULL) MIL.McalFree(cal);  
}
```

4.1.1 Remplissage d'images

Ensuite, pour remplir un buffer MIL à partir d'un buffer `byte[]` il suffira d'appeler dans `Run()` :

```
// byte[] buffer est rempli préalablement...  
MIL.MbufPut2d(_image.ID, 0, 0, numcolumns, numlines, buffer); // et transfere a MIL
```

Le remplissage se fait en *row-major order*, c'est à dire remplissant ligne à ligne. La fonction `MIL.MbufPut2d()` ou son analogue `MIL.MbufPutColor()`, acceptent aussi des buffer `float[]` ou `int16[]`, pour autant qu'il ait été alloué avec les bon format. Si nécessaire on peut copier des buffer d'un type à l'autre avec la fonction `Buffer.BlockCopy()`.

4.1.2 Lecture d'images

On peut accéder à la valeur d'un pixel `x,y` donné avec

```
object r = 0, g = 0, b = 0;  
_image.Get2d(x, y, out r, out g, out b);  
double mono = Convert.ToDouble(r); // Conversion
```

ou alternativement utiliser MIL

```
float[] buf = new float[]; // Allouer buffer selon le format de l'image  
MIL.MbufGet2d(tmp.ID, x, y, 1, 1, buf);
```

Pour lire toute l'image dans un buffer il faudra passer par `MIL.MbufGet2d` ou `MbufGetColor2d`. Ci-dessous un exemple d'extraction de la bande 1, verte :

```
float[] buf = new float[]; // pour image F32  
MIL.MbufGetColor2d(_img.ID, MIL.M_SINGLE_BAND, 1, 0, 0, _img.SizeX, _img.SizeY, buf);
```

4.1.3 Calibration

Pour calibrer une image avec une calibration donnée, il suffira d'appeler dans `Run()`

```
var instance = Matrox.DesignAssistant.Core.Platforms.Platform.Instance;  
instance.Calibrations.ApplyCalibration(_image, calibrationname);
```

Pour voir comment sélectionner la calibration parmi celles disponibles dans la plateforme, voir l'exemple dans la section [5.3](#).

Si par contre on veut simplement hériter de la calibration de l'image en entrée, il suffira de copier son ID :

```
if (_outputImage.CalibrationID != img.CalibrationID)  
    _outputImage.CalibrationID = img.CalibrationID;
```

4.2 Gestion d'erreurs

Pour que des messages d'erreur importants s'affichent dans la fenêtre d'**Execution Messages**, il faut lancer des exceptions de classe **StepExecutionException**.

Dans certains cas, par exemple lors de communication avec des systèmes externes, ce n'est pas clair si c'est avantageux de lancer une exception, car elle implique un saut directe au step Status. Pour laisser le choix à l'utilisateur on peut utiliser un input booléen, p.ex. **ThrowStepExecutionException**, qui décidera le déclenchement de l'exception

```
if (ThrowStepExecutionException)
    throw new StepExecutionException(this, ex.Message);
```

Du côté interface (cf. sec. 5) le nom du input peut être plus spécifique à l'application, comme par exemple *"Error if fails to connect"*.

Il est recommandé que le step retourne un booléen Status pour faciliter la programmation en aval.

5 Programmation de l'interface graphique du custom step

Pour programmer l'interface graphique, dans le projet *UI il faut ouvrir le fichier *UIEditor.xaml et faire les modifications pour placer les inputs.

1. Dans le groupe <Grid.RowDefinitions>, pour chaque input il doit avoir une ligne <RowDefinition Height="Auto"></RowDefinition> plus une ligne <RowDefinition> additionnelle vide, comme dans l'exemple ci-dessous.

```
<Grid.RowDefinitions>
<RowDefinition Height="Auto"></RowDefinition>
<RowDefinition></RowDefinition>
</Grid.RowDefinitions>
```

2. Après le groupe <Grid.RowDefinitions> on trouve directement la configuration des différentes colonnes de chaque ligne, avec le format

```
<TextBlock Grid.Row="0" Grid.Column="0" Text="Action"></TextBlock>
<input:ModernInputBox Grid.Row="0" Grid.Column="1" Expression="{Binding Expressions[Action]}">
</input:ModernInputBox>
<TextBlock Grid.Row="1" Grid.Column="0" Text="PortNumber"></TextBlock>
<input:ModernInputBox Grid.Row="1" Grid.Column="1" Expression="{Binding Expressions[PortNumber]}">
</input:ModernInputBox>
...
```

Pour chaque input du step, on aura un <TextBlock> qui contient le label ou text fixe affiché. Le texte qui s'affichera (nom explicatif de l'input) apparaît dans l'attribut **Text**. Il faudra indiquer sa position avec les attributs **Row** et **Column**.

Pour insérer le contrôle spécial de saisie de l'input il faut ajouter une balise <input:ModernInputBox> dans laquelle on indique le nom de la variable de input souhaitée dans **BindingExpressions[]**

On peut aussi utiliser des **ModernInputCheckBox** et **ModernInputRadioButton**.

Il existe aussi la possibilité de travailler en mode graphique, en ajoutant à les controls au ToolBox de Visual Studio qui se trouvent dans **Matrox.DesignAssistant.UI.dll**. Voir les instructions dans la section **InputBox controls** du document [1].

5.1 Inputs à liste déroulante

Il est possible de faire que un input ait une liste de valeurs possibles, qui sera montrée par une liste déroulante. On peut entrer les valeurs dans un attribut, et les associer à un `enum`.

```
[Input]
[Linkable]
[ConstantValue("Clear", HEArc.StepRSVP.Action.Clear, 0)]
[ConstantValue("AddResponse", HEArc.StepRSVP.Action.AddResponse, 1)]
[ConstantValue("Retrieve", HEArc.StepRSVP.Action.Retrieve, 2)]
[InitialValue("Retrieve")]
public int Action
{
    get { return (int)GetInputValue(nameof(Action)); }
```

5.2 Inputs désactivables ([AvailabilityConstraint])

Il est possible de désactiver (griser) certains inputs en fonction d'autres. Par exemple pour faire certains paramètres disponibles seulement dans un certain mode, à son tour sélectionné dans un autre inputbox. Ceci se fait dans la partie code (pas UI) du step. Il faut déclarer les inputs conditionnellement activables avec l'attribut `[AvailabilityConstraint]` et faire l'override des méthodes `GetAvailability( )` et `GetAvailabilityControllingInputs( )`.

La méthode `bool GetAvailability(string inputName)` retourne, pour chaque nom d'input passé en argument, si il doit être disponible (true) ou grisé (false). Ainsi, elle encode les conditions ou contraintes de disponibilité. Dans l'exemple ci-dessous, les inputs "Format" et "Variable" seront disponibles (non-grisés) seulement lors que la variable Action (liée à l'input "Action") aura la valeur `AddResponse`.

```
protected override bool GetAvailability(string inputName)
{
    bool res = false;
    switch(inputName)
    {
        case nameof(Format):
        case nameof(Variable):
            res = (Action == (int)HEArc.StepRSVP.Action.AddResponse);
            break;
        default:
            res = true;
            break;
    }
    return res;
}
```

La méthode `GetAvailabilityControllingInputs( )` retourne pour chaque nom de input un tableau `IExpression[]` indiquant de quel autre input il dépend (concernant son activation). Par exemple

```
protected override IExpression[] GetAvailabilityControllingInputs(string inputName)
{
    IExpression[] res = null;
    switch (inputName)
    {
        case nameof(Variable): // Variable depends only on Action
        case nameof(Format): // Format depends only on Action
            res = new IExpression[] { InputExpressions[nameof(Action)] };
    }
```

```

        break;
    }
    return res;
}

```

5.3 Listes dynamiques

Pour que un input affiche une liste dynamique de valeurs, typiquement en fonction d'une configuration ou autre, il faut ajouter un *override* de la méthode `string[] GetDynamicValueNames(string inputName)`. Cette méthode retourne un tableau de string avec les valeurs possibles pour un input donné.

Dans l'exemple qui suit elle est utilisée pour retourner les calibrations disponibles sur la plateforme.

```

protected override string[] GetDynamicValueNames(string inputName)
{
    string[] ret = null;
    switch (inputName)
    {
        case "Calibration":
            List<string> r = new List<string>() { "None" };
            var instance = Matrox.DesignAssistant.Core.Platforms.Platform.Instance;
            r.AddRange(instance.Calibrations.GetCalibrationNames().ToList());
            ret = r.ToArray();
            break;
    }
    return ret;
}

```

5.4 Inputs optionnels (None)

Des inputs peuvent avoir la valeur `None`, lorsque par défaut on ne veut pas les utiliser. Pour cela il faut indiquer le input comme

```

[Input]
[Linkable]
[Description("Optional Mask image.")]
[InitialValue("None")]
[ConstantValue("None", ConstantKinds.NullName)]
public Image Mask => (Image)GetInputValue(nameof(Mask));

```

6 Documentation QuickAccess

Une documentation en format XML peut être générée en plaçant dans un dossier `QuickAccess`, au même niveau que le `*.das` et `assets`, un document XML avec le même nom que la classe principale du step. Ce document sera visible dans la fenêtre QuickAccess en haut à droite de l'environnement DA.

```

<?xml version="1.0"?>
<quick-access xsi:noNamespaceSchemaLocation="../../QuickAccess/DesignAssistantQA.xsd"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<section collapsable="true" name="mandatory" text="step Classname" default="expanded">
  <title>Example Step</title>
  <paragraph>The example step performs ...</paragraph>
  <paragraph>It will compute ...</paragraph>
</section>

```



```

<section collapsable="true" name="application" text="Application" default="collapsed">
  <title>Uses and applications</title>
  <paragraph>Example step can be used to ...</paragraph>
</section>
<section collapsable="true" name="inputs" text="Inputs" default="collapsed">
  <title>Step Inputs</title>
  <paragraph>The inputs of the example step are:</paragraph>
  <enumerated-list>
    <item> <paragraph> <input-link target="Input1">Input 1</input-link> is an image that...</paragraph></item>
    <item> <paragraph>Two options:</paragraph>
      <bulleted-list>
        <item> <paragraph> <input-link target="Input2">Input 2</input-link> is checked to...</paragraph></item>
        <item> <paragraph> <input-link target="Input3">Input 3</input-link> is checked to...</paragraph></item>
      </bulleted-list>
    </item>
  </enumerated-list>
</section>
<section collapsable="true" name="outputs" text="Outputs" default="collapsed">
  <title>Step Outputs</title>
  <paragraph>The step provides the following outputs:</paragraph>
  <bulleted-list>
    <item><paragraph>NumImages: The number of processed images in...</paragraph></item>
    <item><paragraph>Name: The name found in ...</paragraph></item>
  </bulleted-list>
</section>
</quick-access>

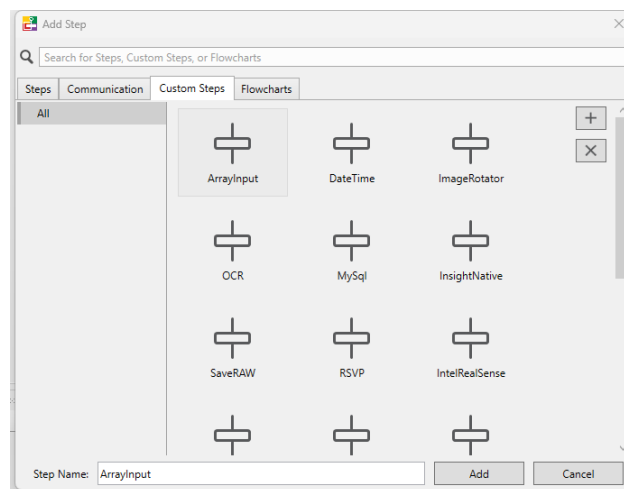
```

On peut intégrer des images avec des balises comme celle-ci

```
<image location="Train.png" layout="inline" height="26"/>
```

7 Utilisation d'un custom step dans DA

Pour ajouter un custom step dans un projet DA, right-clicker **Add step**, comme pour un step normal, et aller au signet **Custom Steps**. Si le custom-step n'est pas dans la liste affichée il faudra l'importer en pressant le bouton **Import Custom Step** et allant chercher le fichier *.das correspondant dans la dossier solution du custom step.



Donner un nom approprié et spécifique lorsqu'on insère le custom step. Comme pour les autres steps, il peut avoir plusieurs instances du même custom step dans un projet.

7.1 Design time vs Runtime

En phase de développement (design time) les DLLs correspondantes au custom step restent dans le dossier où elles sont (dans la solution VS programmée ou reçue d'un tiers.) D'ailleurs, cela pourra empêcher de recompiler la solution, parce que DA détient un *handle* aux DLLs. Il faudra fermer complètement DA (pas seulement le projet) pour pouvoir modifier les DLLs. De l'autre côté, cela veut dire que le custom step ne sera pas sauvegardé dans le projet ; si vous transmettez le projet à votre collègue, vous devrez lui transmettre le custom step séparément.

Lorsqu'on passe en mode runtime (déploiement), les DLLs du custom step seront copiées dans `C:\ProgramData\Matrox Design Assistant\X.Y\Steps\`. C'est pour cela que les DLLs tierces doivent être dans le sous-dossier **assets** déclaré dans ***.das**.

7.2 Débogage d'un custom step

En mode développement le custom step peut être débogué en pas-à-pas avec Visual Studio. Suivre les instructions de la section **Debugging** du document [1].

8 Actualisation d'un custom step de version précédente

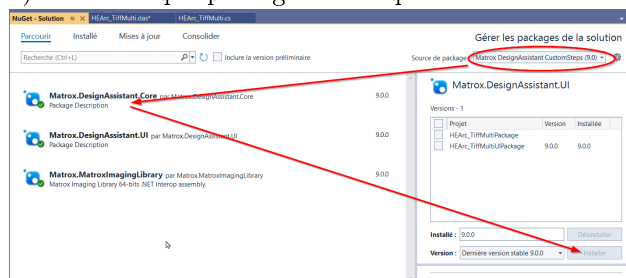
Lorsqu'on change de version de Matrox DA souvent on ne pourra pas utiliser les custom steps faits pour une version précédente. Dans le meilleur des cas, il sera nécessaire de recompiler la solution avec les bibliothèques (DLL) correspondantes à la nouvelle version. Pour cela il faudra changer dans la solution les éléments suivants :

— Dans chaque projet il faut faire pointer les Références ou Dépendances vers les DLLs

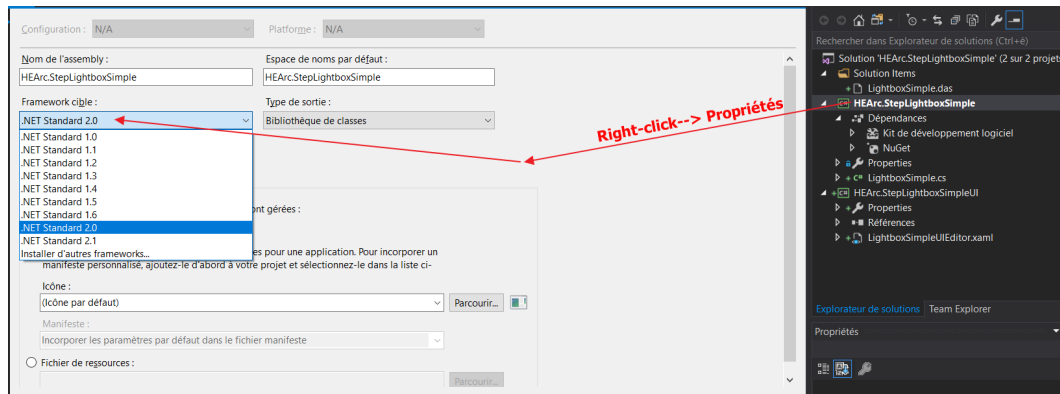
1. `Matrox.DesignAssistantCore.dll`
2. `Matrox.MatroxImagingLibrary.dll`
3. `Matrox.DesignAssistantUI.dll` (seulement pour les projets UI)

présentes dans le dossier `C:/Program Files/Matrox Imaging/DA */CustomSteps/References` de l'installation correspondante.

Pour cela il faudra effacer les anciennes références et ajouter les nouvelles à leur place. On peut aussi les actualiser avec *NuGet*, avec **File→Outils →Gestionnaire de package NuGet →Gérer des packages pour la solution**. En haut à droite, changer le menu déroulant **Source de package** à la valeur DA actuelle (DA9) et sur chaque package listé cliquer le bouton **Installer**.



— Pour chaque projet il faut configurer la version cible du .NET Framework utilisé par Matrox DA. Normalement le message d'erreur à la compilation indique la version cible à utiliser. Pour configurer les cibles, sur chaque projet il faut faire **right-click** → **Propriétés** et sélectionner le "Framework cible" désiré. S'il n'est pas disponible on peut l'installer en cliquant tout en bas de la liste déroulante.



Références bibliographiques

- [1] I. LESZKOWICZ, "Introduction to Matrox Design Assistant Custom Steps.pdf," 8 May 2019. adresse : [/c:/Program%20Files/Matrox%20Imaging/DA%209.0/CustomSteps/Introduction%20to%20Matrox%20Design%20Assistant%20Custom%20Steps.pdf](#).
- [2] "Design Assistant Custom Step SDK.chm," Matrox. adresse : [/c:/Program%20Files/Matrox%20Imaging/DA%209.0/CustomSteps/Design%20Assistant%20Custom%20Step%20SDK.chm](#).

A Troubleshooting

A.1 Error : Dispose was not called on a *StepX* instance

Au moment de deployer un projet qui fonctionne en mode développement le message "Dispose was not called on a *StepX* instance" s'affiche, suivi de "The project cannot run because an error occurred during the load of one or more steps: StepX", où StepX sera un custom step.

Le problème apparaît parce que DA n'arrive pas à charger une DLL qui est nécessaire et qui manque dans le fichier *.das.

A.1.1 Solution

Compléter le fichier *.das avec toutes les DLLs nécessaires. Souvent il s'agira d'une DLL appelée par la DLL principale. Pour identifier les dépendances entre DLLs on peut utiliser l'utilitaire [Dependencies](#) (voir option download binaries).

S'il y a eu un renommage important dans le projet Visual Studio, il faudra enlever l'ancien custom step, aussi dans C:\ProgramData\Matrox Design Assistant\9.0 \Steps et importer le nouveau.

A.2 Error : Error loading assembly

Lors de l'ajout (Add) d'un CustomStep (même s'il a été importé dans DA avec succès), une fenêtre d'erreur apparaît : "Error loading Assembly".

Le problème apparaît parce que DA n'arrive pas à charger une DLL qui est nécessaire et qui manque dans le fichier *.das.

A.2.1 Solution

S'assurer que le fichier *.das contient le chemin de toutes les DLLs requises par le custom Step. On peut utiliser Dependencies (<https://github.com/lucasg/Dependencies>) pour le vérifier.

Il pourrait s'agir d'une DLL de système, en particulier si le custom step est compilé contre `netstandard2.0`. Dans ce cas-la il faut installer le redistributable plus récent (NetCore 3.1) depuis <https://dotnet.microsoft.com/en-us/download/visual-studio-sdks>